Aula 10
Linguagem SQL - Complementos

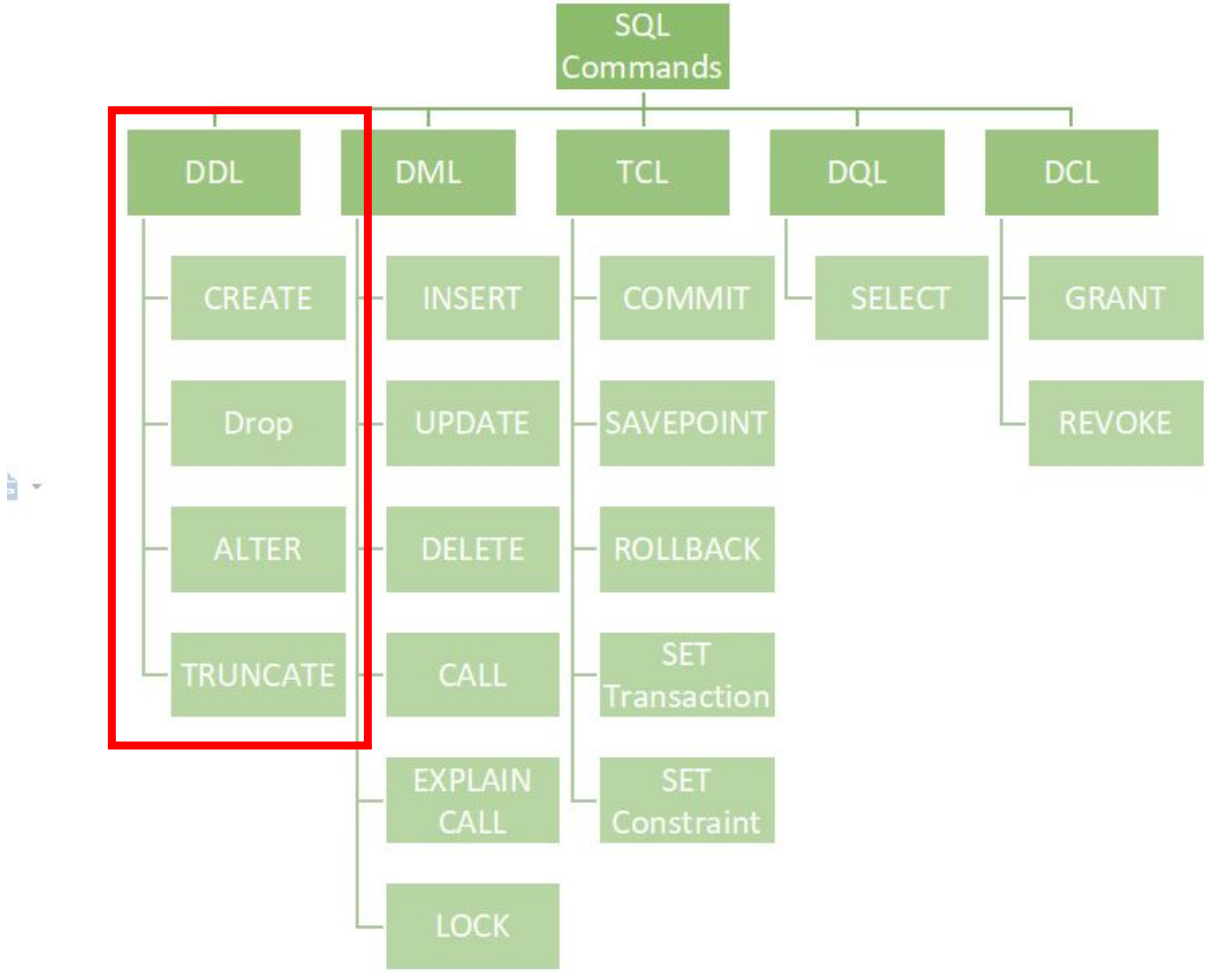
Bruno Elias Penteado

bruno.penteado@unesp.br

Na aula anterior...

- Ferramenta CASE
  - ERD Tool (pgAdmin)
- Revisão SQL

Categorias de comandos SQL



Atividade

- Projeto (16/09) – via Classroom:
  - Grupos de 4 pessoas
  - Minimundo definido pelo grupo (entre 10-15 entidades)
  - Documento com 3 partes:
    - **Modelo conceitual**: explicação sobre o minimundo, regras de negócio e DER
    - **Modelo lógico**: diagrama lógico já normalizado, com possíveis justificativas
    - **Modelo físico**: arquivo .sql comentado
  - Crie um script .sql (arquivo) que:
    - Crie um novo esquema (com o nome de seu projeto)
    - Crie as tabelas (aplique todas as restrições (*constraints*) necessárias)

Atividade no Classroom

- Abra o **pgAdmin;**
- Execute o script da última aula para criar um novo banco de dados, já populando-o;
- As próximas atividades usarão este novo banco de dados (schema *loja_aula1*)

Agenda

- **SQL Intermediário**
  - DQL
    - **Window functions**
    - CTE / Recursividade
    - Trigger
    - Functions
    - Procedures e cursores
  - DCL
  - TCL

Funções de janela

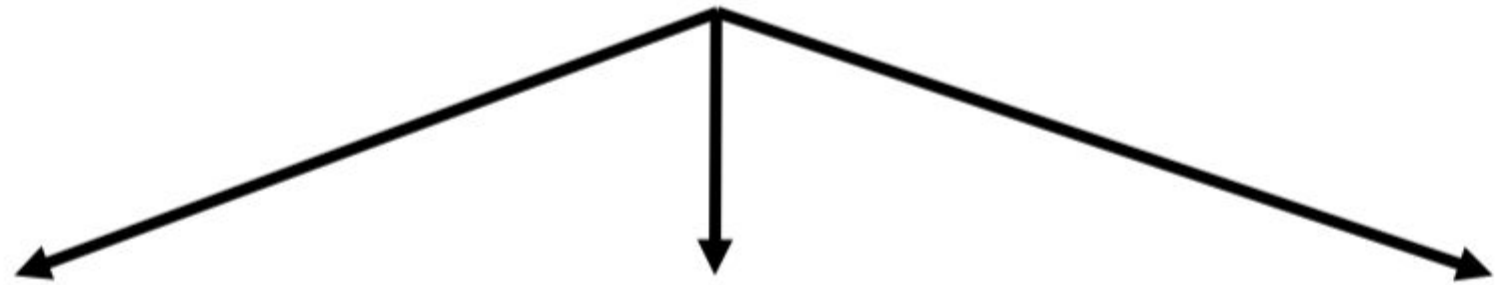
- Uma **função de janela** realiza um cálculo considerando um conjunto de linhas relacionadas à linha atual, mas **sem agrupar ou eliminar essas linhas**

- Essa é a diferença principal em relação ao GROUP BY:
  - GROUP BY reúne várias linhas e produz uma linha por grupo;
  - uma função de janela calcula valores sobre um grupo, mas mantém cada linha original no resultado.

Funções de janela

- Funções de janela são especialmente úteis para:
  - calcular rankings;
  - numerar linhas;
  - comparar uma linha com a média do seu grupo;
  - calcular totais acumulados;
  - calcular percentuais de participação;
  - acessar valores da linha anterior ou seguinte;
  - selecionar os primeiros registros de cada grupo;
  - calcular médias móveis.

Window Functions



Aggregate	Ranking	Value
AVG()	ROW_NUMBER	LAG()
MAX()	RANK()	LEAD()
MIN()	DENSE_RANK()	FIRST_VALUE()
SUM()	PERCENT_RANK()	LAST_VALUE()
COUNT()	NTILE()	NTH_VALUE()

Funções de janela

```
SELECT *
    ,ROW_NUMBER() OVER (ORDER BY Sales DESC) AS ROW_NUMBER
    ,RANK() OVER (ORDER BY Sales DESC) AS RANK
    ,DENSE_RANK() OVER (ORDER BY Sales DESC) AS DENSE_RANK
    ,NTILE(3) OVER(ORDER BY Sales DESC) AS NTILE
FROM Example_Ranking
ORDER BY Sales DESC;
```

100 %

Results | Messages

	ID	Product	Product_Group	Sales	ROW_NUMBER	RANK	DENSE_RANK	NTILE
1	3	Wardrobe	Kitchen	410	1	1	1	1
2	6	Wash Machine	Bathroom	400	2	2	2	1
3	4	Bed	Bedroom	200	3	3	3	2
4	2	Table	Kitchen	200	4	3	3	2
5	1	Chair	Kitchen	110	5	5	4	3
6	5	Bath	Bathroom	100	6	6	5	3

Funções de janela

- Os principais componentes são:
  - **OVER**: indica que a função será utilizada como função de janela;
  - **PARTITION BY**: divide as linhas em grupos, sem agrupá-las em uma única linha;
  - **ORDER BY**: define a ordem das linhas dentro de cada grupo.

Funções de janela

- *Exemplo 1*: mostrar cada produto e também o preço médio de todos os produtos.

```
SELECT
    nome,
    preco,
    ROUND(AVG(preco) OVER (), 2) AS medi
FROM produto
ORDER BY preco DESC;
```

nome	preco	media_geral
Notebook Pro 14	4800.00	688.13
Monitor 24 polegadas	1200.00	688.13
Cadeira Ergonomica	950.00	688.13
Mesa para Escritorio	800.00	688.13
Caixa de Som	500.00	688.13
SSD 1 TB	450.00	688.13
Teclado Mecanico	400.00	688.13

Esta expressão calcula a média usando todas as linhas. Os parênteses vazios de **OVER( )** indicam que não há divisão em grupos.

```
SELECT
                                                    ia_geral
```

nome	preco	media_geral
Notebook Pro 14	4800.00	4800.000000
Monitor 24 polegadas	1200.00	1200.000000
Cadeira Ergonomica	950.00	950.000000
Mesa para Escritorio	800.00	800.000000
Caixa de Som	500.00	500.000000
SSD 1 TB	450.00	450.000000
Teclado Mecanico	400.00	400.000000

Funções de janela

- *Exemplo 2*: ranquear os Produtos, dos mais caros para os mais baratos

```
SELECT
    nome,
    preco,
    ROW_NUMBER() OVER (ORDER BY preco
    RANK()       OVER (ORDER BY preco
    DENSE_RANK() OVER (ORDER BY preco
FROM produto;
```

	nome	preco	numero	ranking	ranking_denso
▶	Notebook Pro 14	4800.00	1	1	1
	Monitor 24 polegadas	1200.00	2	2	2
	Cadeira Ergonomica	950.00	3	3	3
	Mesa para Escritorio	800.00	4	4	4
	Caixa de Som	500.00	5	5	5
	SSD 1 TB	450.00	6	6	6
	Teclado Mecanico	400.00	7	7	7
	Mochila para Notebook	400.00	8	7	7
	Fone Bluetooth	350.00	9	9	8
	Webcam HD	350.00	10	9	8
	Controle sem Fio	320.00	11	11	9

Result 48

Funções de janela

- *Exemplo 2*: ranquear os Produtos, dos mais caros para os mais baratos, por Categoria

```
SELECT id_categoria, nome, pre
    RANK() OVER (
        PARTITION BY id_categ
        ORDER BY preco DESC
    ) AS posicao_na_categoria
FROM produto
ORDER BY id_categoria, posicac
```

id_categoria	nome	preco	posicao_na_categoria
1	Notebook Pro 14	4800.00	1
1	Monitor 24 polegadas	1200.00	2
1	SSD 1 TB	450.00	3
2	Cadeira Ergonomica	950.00	1
2	Mesa para Escritorio	800.00	2
2	Luminaria LED	150.00	3
3	Caixa de Som	500.00	1
3	Fone Bluetooth	350.00	2
3	Webcam HD	350.00	2
4	Teclado Mecanico	400.00	1
4	Mochila para Notebook	400.00	1

Result 49

Funções de janela

- Síntese das funções:

Necessidade	Solução
Cada produto acompanhado da média da categoria	*AVG() OVER (PARTITION BY ...)*
Numerar linhas	*ROW_NUMBER()*
Criar ranking	*RANK() ou DENSE_RANK()*
Acumular valores	*SUM() OVER (ORDER BY ...)*

Funções de janela

- Pergunta central: ***O cálculo deve resumir as linhas ou deve aparecer ao lado de cada linha?***
  - Se deve resumir, normalmente usamos GROUP BY. Se deve preservar as linhas, geralmente uma função de janela é mais apropriada.

Agenda

- **SQL Intermediário**
  - DQL
    - Window functions
    - **CTE / Recursividade**
    - Trigger
    - Functions
    - Procedures e cursores
  - DCL
  - TCL

Common Table Expression (CTE)

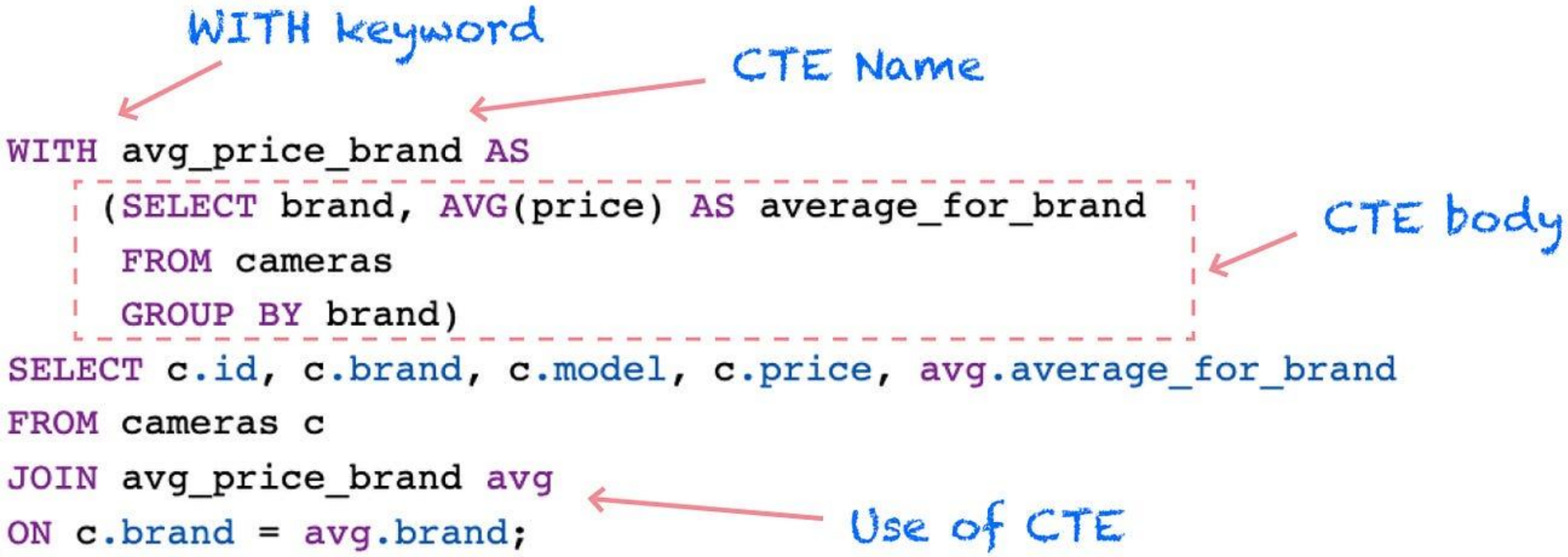
- Consulta temporária que recebe um nome e pode ser utilizada por outra consulta
- Operador WITH permite definir uma tabela que será usada somente em uma consulta específica
- Desejável quando uma consulta possui várias etapas intermediárias e começa a ficar difícil de compreender, testar ou manter

```
;WITH OrdersByMonth2020
AS
(
    SELECT DateName(month, OrderDate) AS OrderMonth
    FROM Orders
    WHERE YEAR(OrderDate) = 2020
)
SELECT OrderMonth, COUNT(*) AS NumberOfOrders
From OrdersByMonth2020
GROUP BY OrderMonth
```

Results | Messages

OrderMonth	NumberOfOrders
August	2
October	4
September	5

Common Table Expression (CTE)



Common Table Expression (CTE)

- *Exemplo 1*: recupere os dados apenas dos Produtos caros (*preco* > 500 reais)

```sql
SELECT nome, preco
FROM produto
WHERE preco > 500;
```

=

```sql
WITH produtos_caros AS (
    SELECT
        nome,
        preco
    FROM produto
    WHERE preco > 500
)
SELECT *
FROM produtos_caros;
```

Common Table Expression (CTE)

- *Exemplo 2*: calcule a quantidade média de produtos por categoria

```sql
SELECT AVG(quantidade) AS media_produtos
FROM (
    SELECT
        id_categoria,
        COUNT(*) AS quantidade
    FROM produto
    GROUP BY id_categoria
) AS produtos_por_categoria;
```

=

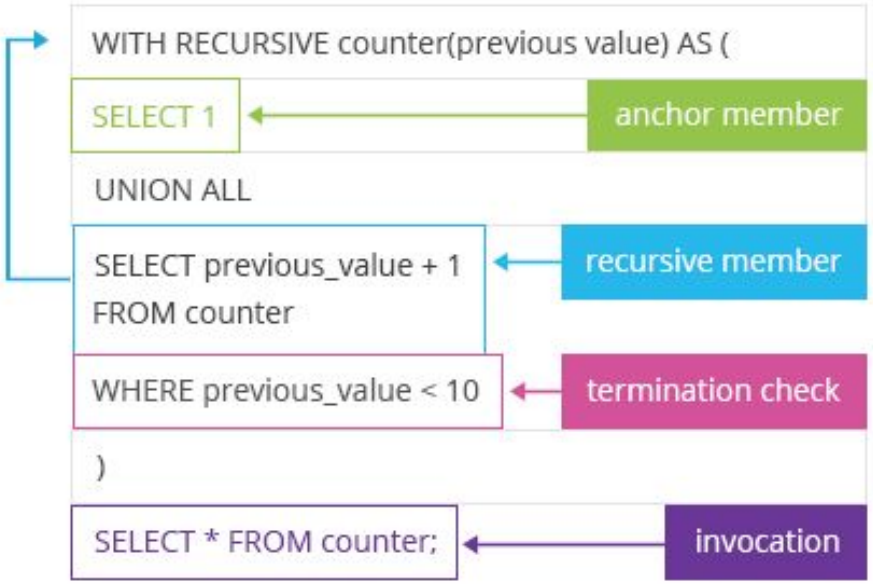
```sql
WITH produtos_por_categoria AS (
    SELECT
        id_categoria,
        COUNT(*) AS quantidade
    FROM produto
    GROUP BY id_categoria
)
SELECT
    AVG(quantidade) AS media_produtos
FROM produtos_por_categoria;
```

Common Table Expression (CTE)

- WITH
  - Trabalho pesado de organização, limpeza e redução de dados
  - Objetivo de criar uma “tabela limpa” para processar depois
    - Filtros WHER
    - Agregações
    - Window functions
    - JOINs complexos
- SELECT
  - Toque final: filtros adicionais, junções complementares, ordenação, paginação

Common Table Expression (CTE)

- Em certos casos, precisamos executar um comando **recursivamente**
  - Processar hierarquias (pastas/subpastas; funcionários/supervisores)
  - Processar grafos
- Estrutura básica:



Common Table Expression (CTE)

- *Exemplo 3*: mostre a categoria “Tecnologia” e todos seus descendentes

```
WITH RECURSIVE arvore_categorias AS (
    -- Caso inicial: categoria escolhida como raiz da busca.
    SELECT  id_categoria, nome, id_categoria_pai, 0 AS nivel
    FROM categoria
    WHERE nome = 'Tecnologia'

    UNION ALL

    -- Passo recursivo: encontra as filhas da categoria anterior.
    SELECT  c.id_categoria, c.nome, c.id_categoria_pai, ac.nivel + 1
    FROM categoria AS c
    INNER JOIN arvore_categorias AS ac
        ON c.id_categoria_pai = ac.id_categoria
```

id_categoria	nome	id_categoria_pai	nivel
7	Tecnologia	NULL	0
1	Informatica	7	1
6	Games	7	1
4	Acessorios	1	2

```
egoria, nome, id_categoria_pai, nivel
e_categorias
ivel, id_categoria;
```

Agenda

- **SQL Avançado**
  - DQL
    - Window functions
    - CTE / Recursividade
    - **Trigger**
    - Functions
    - Procedures e cursores
  - DCL
  - TCL

Trigger

- Conceito de gatilho
- Como funcionam eventos no banco de dados
- Criar triggers no MySQL
- Cenários reais

Triggers

- Ações engatilhadas que disparam automaticamente quando um evento específico ocorre em uma tabela ou visão.
- São funções executadas pelo SGBD sem intervenção direta do usuário após o disparo do evento
- Pode ser útil para especificar uma condição que, se violada, causa alguma ação monitora o estado do banco de dados

Triggers

- Às vezes precisamos reagir quando os dados do banco forem alterados
- Como resolver?
  - Automatizando essas ações, sem necessitar de chamada explícita

Trigger

Um trigger executa automaticamente uma ação quando um evento ocorre.

O que é um trigger?

- Bloco de código executado de forma automática pelo banco de dados em resposta aos seguintes eventos em uma tabela:
  - INSERT
  - UPDATE
  - DELETE

Fluxo de funcionamento

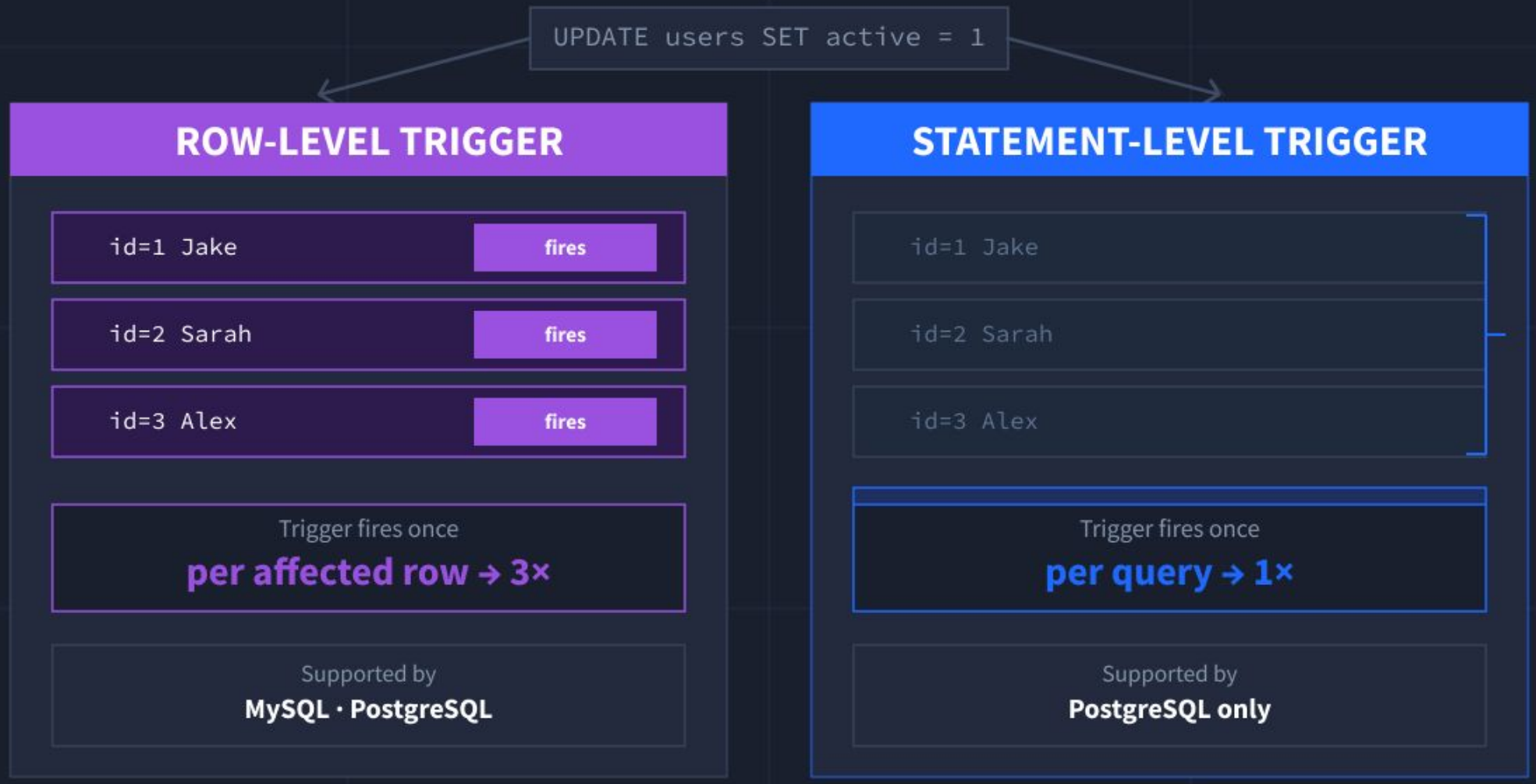
- Evento Timing Ação

- Evento: INSERT, UPDATE, DELETE
- Timing: BEFORE, AFTER, INSTEAD OF
- Ação: seu código SQL

Timing

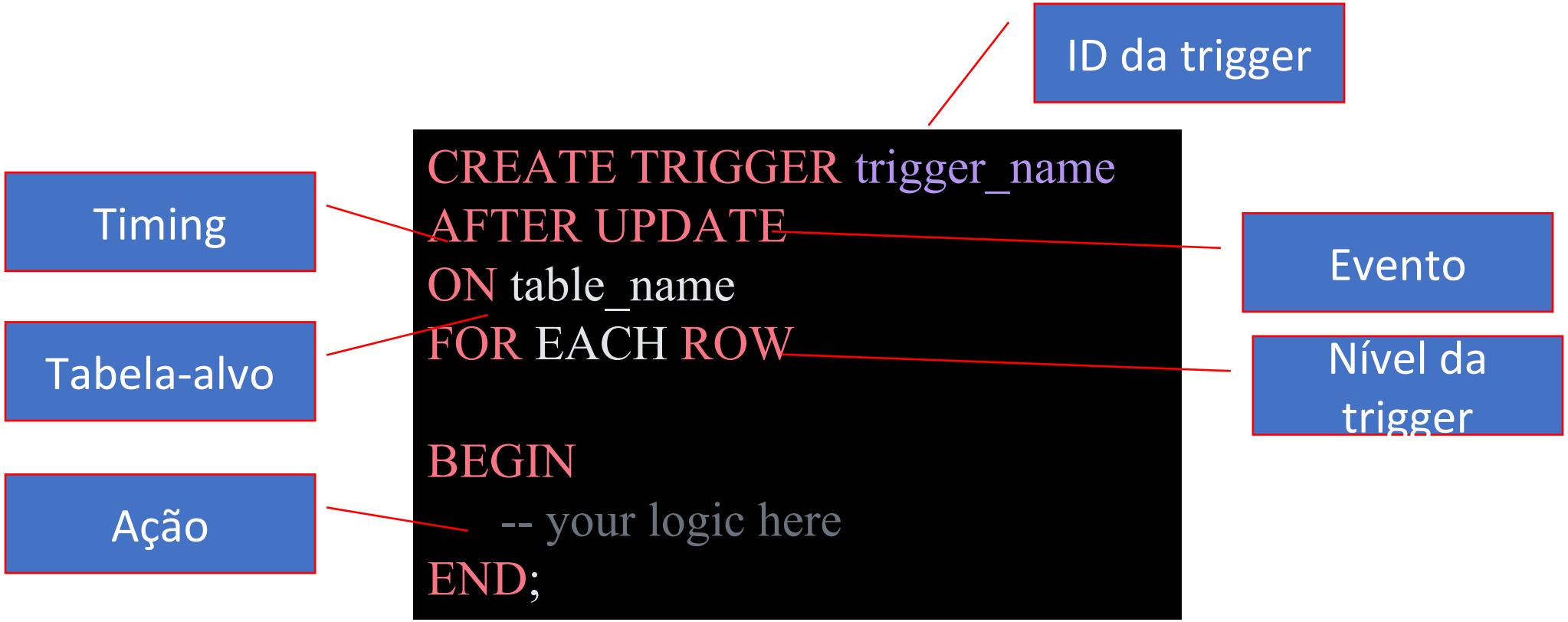
- **BEFORE**: dispara a trigger antes que alteração seja escrita no banco de dados útil para validações
- **AFTER**: dispara a trigger depois que a alteração foi feita útil para criar logs de auditoria ou sincronizar tabelas relacionadas
- INSTEAD OF: dispara a trigger no lugar da operação original útil para manutenção de tabelas legadas, soft delete, particionamento, criptografar dados sensíveis

Row-Level vs Statement-Level



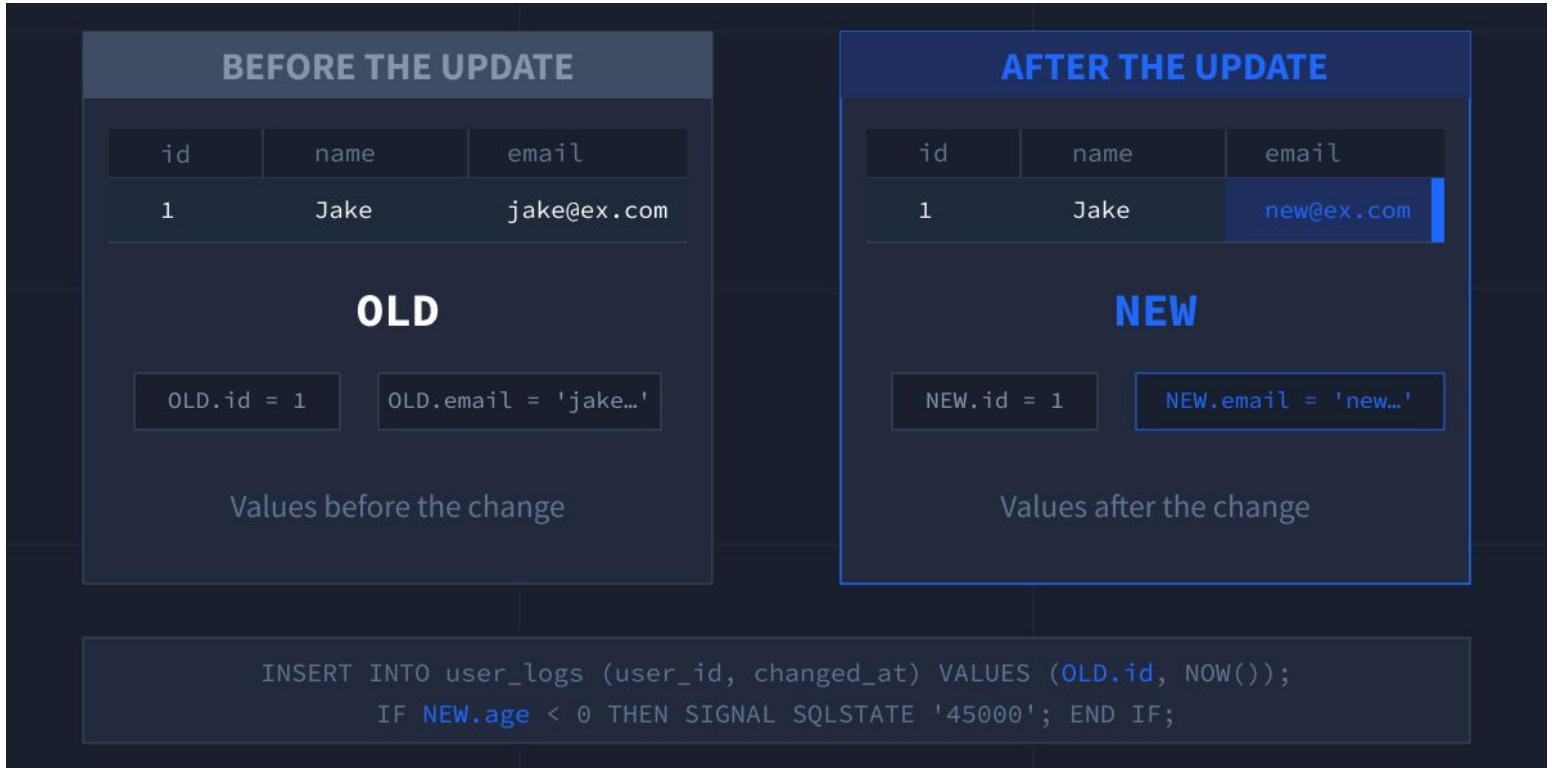
Sintaxe

- Como criar no MySQL



UPDATE

- Quando uma tupla é atualizada temos acesso aos valores:
  - NEW: valor após a alteração
  - OLD: valor antes da alteração



Trigger

- Exemplo no MySQL Workbench (arquivo no *Classroom*)

- Crie uma tabela Usuario e uma Usuario_log
  - Quando alguém editar dados de um usuário, crie um log

Escopo

- Comandos DML (INSET, UPDATE, DELETE)
- Comandos DDL (não suportado no MySQL): monitora eventos na estrutura do banco de dados
  - CREATE, ALTER, DROP
  - Ex.: impedir usuários de excluírem tabelas em ambiente de produção, salvando quem fez a tentativa em uma tabela de auditoria

Agenda

- **SQL Avançado**
  - DQL
    - Window functions
    - CTE / Recursividade
    - Trigger
    - **Functions**
    - Procedures e cursores
  - DCL
  - TCL

Functions

- Uma função é um bloco de código nomeado e reutilizável que fica salvo no banco de dados, aceita dados de entrada e retorna um valor como resultado
- Útil para:
  - Reuso de código: evita repetir a mesma lógica em vários lugares;
  - Simplifica consultas: deixa comandos SELECT mais fáceis de ler;
  - Centralizar regras: mantém regras de negócio no próprio BD, garantindo que o resultado seja sempre o mesmo para qualquer aplicação acessando os dados

Functions

- Funções embutidas no MySQL:
  - String: CONCAT() / LOWER () / UPPER () / REPLACE() / SUBSTRING()
  - Numérica: ROUND() / CEIL () / FLOOR() / ABS()
  - Datas: NOW() / CURDATE() / DATEDIFF()
  - Agregação: COUNT() / MAX () / MIN () / AVG ()
  - Controle: IF () / COALESCE ()

Functions

- *User-defined functions* (UDF): função definida pelo usuário para realizar uma tarefa específica que as funções nativas de um SGBD não conseguem resolver sozinhas

Funções

- Sintaxe

```
CREATE [OR REPLACE] FUNCTION nome_da_funcao (
    parametro1 tipo_dado,
    parametro2 tipo_dado
)
RETURNS tipo_retorno AS $$
DECLARE
    -- Aqui você declara as variáveis que vai usar (opcional)
    minha_variavel tipo_dado;
BEGIN
    -- Aqui fica a lógica da sua função
    minha_variavel := parametro1 + parametro2;

    -- Toda função obrigatoriamente precisa retornar algo
    RETURN minha_variavel;
END;
$$ LANGUAGE plpgsql;
```

Funções

- Exemplo

```
CREATE OR REPLACE FUNCTION calcular_desconto(
    preco_original NUMERIC,
    porcentagem_desconto NUMERIC
)
RETURNS NUMERIC AS $$
DECLARE
    valor_desconto NUMERIC;
    preco_final NUMERIC;
BEGIN
    -- 1. Calcula quanto vale o desconto em dinheiro
    valor_desconto := (preco_original * porcentagem_desconto) / 100;

    -- 2. Subtrai o desconto do preço original
    preco_final := preco_original - valor_desconto;

    -- 3. Retorna o preço final arredondado com 2 casas decimais
    RETURN ROUND(preco_final, 2);
END;
$$ LANGUAGE plpgsql;
```

```
SELECT
    nome_produto,
    preco,
    calcular_desconto(preco, 10) AS preco_com_desconto
FROM produtos;
```

Trigger

- Exemplo no MySQL Workbench (arquivo no *Classroom*)

- Crie uma UDF ***fn_CalcularDesconto (preco_original, desconto_percentual)***
  - Retorna o novo valor, já com o desconto

Agenda

- **SQL Avançado**
  - DQL
    - Window functions
    - CTE / Recursividade
    - Trigger
    - Functions
    - **Procedures e cursores**
  - DCL
  - TCL

Procedures

- *Stored procedure* (procedimento armazenado)
- Bloco de código salvo no BD e que pode ser executado quando necessário
- Pode retornar conjunto de tuplas
- Vantagens:
  - Reuso (mesma lógica em vários lugares)
  - Desempenho (única chamada ao BD)
  - Segurança (acesso a ações e não a dados)
  - Organização (centraliza regras de negócio)

```
CREATE OR REPLACE PROCEDURE aplicar_desconto_no_produto(
    id_do_produto INT,
    porcentagem_desconto NUMERIC
)
AS $$
BEGIN
    -- Executa uma ação de alteração (UPDATE)
    UPDATE produtos
    SET preco = ROUND(preco * (1 - porcentagem_desconto / 100), 2)
    WHERE id = id_do_produto;

    -- Opcional: Você pode salvar a alteração na hora
    COMMIT;
END;
$$ LANGUAGE plpgsql;
```

```
CALL aplicar_desconto_no_produto(5, 10);
```

Cursor

- Objeto do BD para navegar e processar, linha por linha, o resultado de uma instrução SELECT
- Funciona como um ponteiro: permite que o sistema leia ou altere os valores daquela linha antes de passar para a próxima
- Só pode ser usado em *functions*, *procedures* ou *triggers*

```
CREATE PROCEDURE curdemo()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE a CHAR(16);
  DECLARE b, c INT;
  DECLARE cur1 CURSOR FOR SELECT id,data FROM test.t1;
  DECLARE cur2 CURSOR FOR SELECT i FROM test.t2;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

  OPEN cur1;
  OPEN cur2;

  read_loop: LOOP
    FETCH cur1 INTO a, b;
    FETCH cur2 INTO c;
    IF done THEN
      LEAVE read_loop;
    END IF;
    IF b < c THEN
      INSERT INTO test.t3 VALUES (a,b);
    ELSE
      INSERT INTO test.t3 VALUES (a,c);
    END IF;
  END LOOP;

  CLOSE cur1;
  CLOSE cur2;
END;
```

Declarar cursor

Abrir cursor

Recuperar dados

Processar dados

Fechar cursor

Exemplo em MySQL

Agenda

- **SQL Avançado**
  - DQL
    - Window functions
    - CTE / Recursividade
    - Trigger
    - Functions
    - Procedures e cursores
  - **DCL**
  - TCL

DCL (*Data Control Language*)

- Comandos para atribuir ou revogar permissões a usuários específicos ou grupos de usuários

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions, or functions.	`GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];`
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	`REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];`

DCL - Exemplo

```
GRANT privilege
ON privilege_level TO account_name;
```

```
-- Criando um grupo para o time de vendas
CREATE ROLE grupo_vendas;

-- Dando permissões para esse grupo
GRANT SELECT, INSERT ON tabelas_vendas TO grupo_vendas;
```

```
-- Criando o usuário para a Ana
CREATE USER ana WITH PASSWORD 'senha_segura';

-- Colocando a Ana dentro do grupo de vendas
GRANT grupo_vendas TO ana;
```

No PosgreSQL: ROLE organiza permissões mas sem poder fazer login

Exemplo

Você é administrador do sistema de uma loja e precisa criar um usuário para um funcionário do setor de vendas que só pode consultar (SELECT) e inserir (INSERT) dados na tabela de Produtos.

- Criar usuário

```
-- Cria um usuário que pode logar e define uma senha segura
CREATE USER analista_dados WITH PASSWORD 'SenhaSegura123!';
```

- Conceder permissões

```
-- Primeiro, o usuário precisa de permissão para se conectar ao banco de dados
GRANT CONNECT ON DATABASE seu_banco_de_dados TO analista_dados;

-- Também é necessário dar acesso ao schema (geralmente usamos o 'public')
GRANT USAGE ON SCHEMA public TO analista_dados;

-- Agora, concedemos permissão de leitura (SELECT) e inserção (INSERT) na tabela
GRANT SELECT, INSERT ON TABLE public.vendas TO analista_dados;
```

- Verificar permissões
- Remover permissões

Exemplo

Você é administrador do sistema de uma loja e precisa criar um usuário para um funcionário do setor de vendas que só pode consultar (SELECT) e inserir (INSERT) dados na tabela de Produtos.

- Criar usuário
- Conceder permissões
- Verificar permissões
- Remover permissões

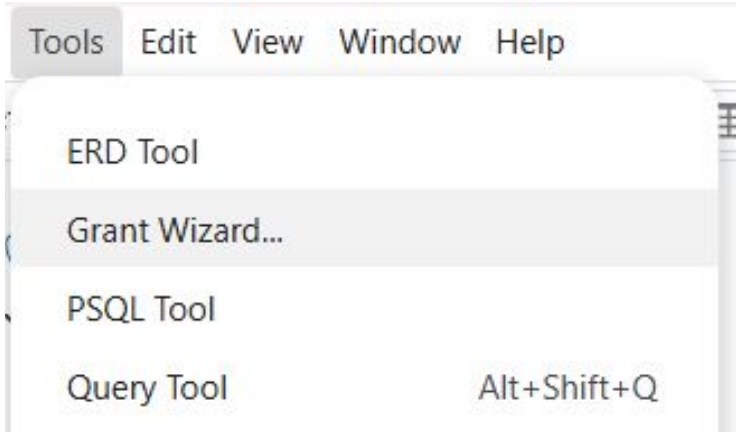
```sql
SELECT
    grantee AS usuario,
    privilege_type AS permissao,
    table_name AS tabela
FROM
    information_schema.role_table_grants
WHERE
    grantee = 'analista_dados';
```

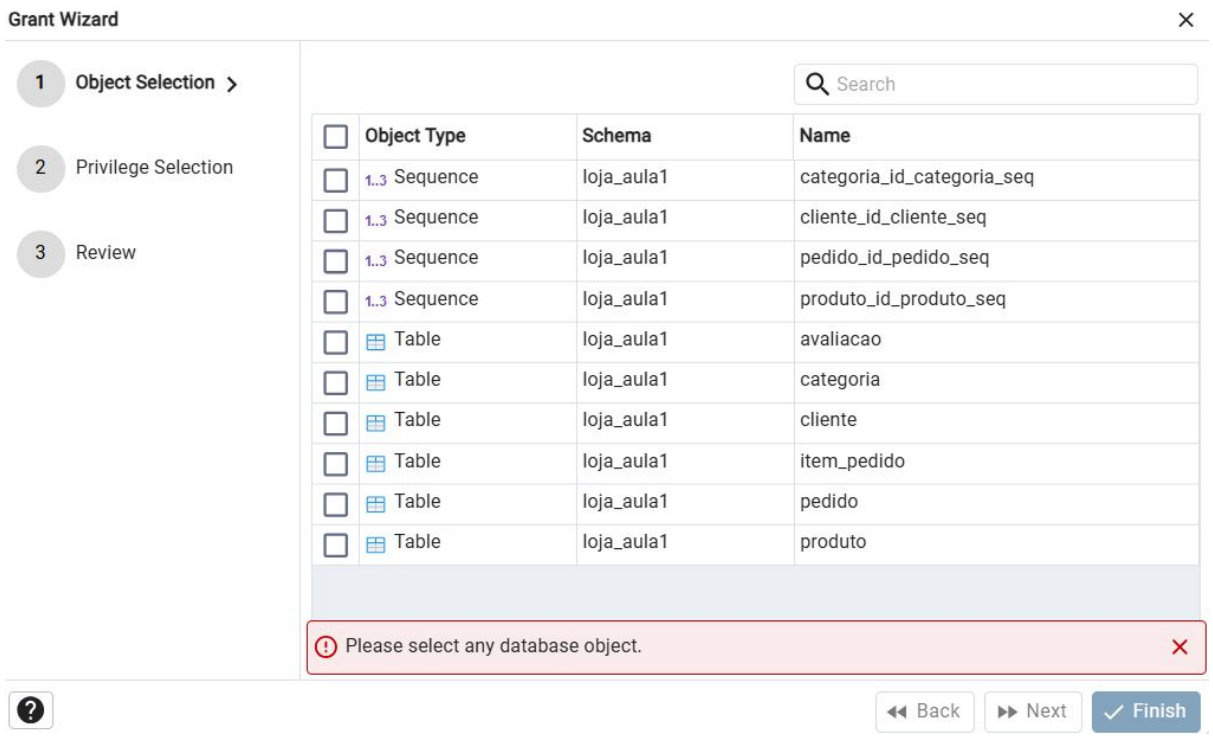
```sql
-- Remove apenas a permissão de inserir novos dados (mantém a de leitura)
REVOKE INSERT ON TABLE public.vendas FROM analista_dados;

-- Remove todas as permissões restantes na tabela de uma só vez
REVOKE ALL PRIVILEGES ON TABLE public.vendas FROM analista_dados;
```

DCL - Workbench

- pgAdmin
  - Menu ‘Tools > *Grant Wizard...*’ (um schema deve estar selecionado)





Agenda

- **SQL Avançado**
  - DQL
    - Window functions
    - CTE / Recursividade
    - Trigger
    - Functions
    - **Procedures e cursores**
  - DCL
  - **TCL**

Comandos TCL

- *Transaction Control Language*
- Transações agrupam conjunto de comandos em **uma única unidade lógica** de execução
- Cada transação começa com uma tarefa e **termina quando todas as tarefas no grupo foram completadas com sucesso**
- Caso qualquer uma das tarefas falhe, **a transação toda falha** (nenhum dos comandos é efetivado)
- 2 possíveis resultados: sucesso ou falha

Comandos TCL

Comando	Descrição	Sintaxe
BEGIN TRANSACTION	Começa uma nova transação	BEGIN TRANSACTION [transaction_name];
COMMIT	Salva todas as mudanças durante a transação	COMMIT;
ROLLBACK	Desfaz todas as modificações feitas durante a transação	ROLLBACK;
SAVEPOINT	Cria um ponto de salvamento dentro da transação atual	SAVEPOINT savepoint_name;

Comandos TCL

- Exemplo

```
BEGIN;

UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
UPDATE contas SET saldo = saldo + 100 WHERE id = 2;

-- Se algo der errado, você pode usar: ROLLBACK;
-- Se estiver tudo correto, você confirma:
COMMIT;
```

Comandos TCL

- Exemplo

```
-- 1. Inicia a transação
BEGIN;

-- 2. Faz uma alteração correta
UPDATE produtos SET preco = 10.00 WHERE id = 1;

-- 3. Cria o primeiro ponto de salvamento
SAVEPOINT pos_produto_1;

-- 4. Faz outra alteração correta
UPDATE produtos SET preco = 25.00 WHERE id = 2;

-- 5. Cria o segundo ponto de salvamento
SAVEPOINT pos_produto_2;

-- 6. OOPS! Você cometeu um erro e alterou o produto errado
UPDATE produtos SET preco = 0.00 WHERE id = 999;

-- 7. Desfaz APENAS o erro voltando para o segundo ponto
ROLLBACK TO SAVEPOINT pos_produto_2;

-- 8. Finaliza salvando os produtos 1 e 2 (o erro foi ignorado!)
COMMIT;
```

Atividade

- Crie uma **CTE** que calcule o preço médio dos produtos de cada categoria. Em seguida, liste os produtos cujo preço seja superior à média de sua própria categoria. Apresente o produto, seu preço e a média da categoria.

- (***window function***)Apresente os pedidos em ordem cronológica e calcule o frete acumulado. Fretes não informados devem ser considerados iguais a zero. Mostre o código do pedido, a data, o frete e o frete acumulado.

- Crie uma **função** *fn_total_pedido* que recebe o id de um Pedido e retorna o valor total deste Pedido
  - A soma dos subtotais de todos os itens + o frete (tratando NULL como 0)
  - Teste chamando a função com 2 pedidos diferentes (frete NULL e NOT NULL)

- Crie um **usuário** do banco de dados chamado *relatorios* com permissão apenas de SELECT em todas as tabelas do schema
  - Create user > grant connect > grant usage > grant select

Seminários

Seminários BD:

- Chave-valor [**Giovana**]
- Documentos [**Isaac**]
- Colunar **[Mariana]**
- Geoespacial [**David**]
- Vetorial [**João**]
- Grafo propriedades [**Mateus**]
- Grafo RDF [**Felipe**]
- Eventos **[Enrico]**

~~• Séries temporais~~

~~• Multimodal~~

Para cada um (20 min):

- Quando surgiu
- Características do modelo de dados (como organiza os dados)
- Vantagens e limitações
- Aplicações comuns
- Exemplos de SGBDs comerciais
- Executar um exemplo

Referências bibliográficas

- Notas de aulas, prof. José Eduardo Santarém Segundo. **Banco de Dados - Normalização - Parte 1**. UNIVESP, Eixo de Computação. Disponível em: https://www.youtube.com/watch?v=OsebNaYQpk8
- ELMASRI, Ramez; NAVATHE, Shamkant B**. Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson Universidades, 2019. 1152 p. ISBN-13 978-8543025001.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de Banco de Dados.** 7. ed. Rio de Janeiro: GEN LTC, 2020. 754 p. ISBN-13 978-8595157330.
- DATE, Christopher J. Introdução a Sistemas de Bancos de Dados. GEN LTC, 2004
- Notas de aulas, prof. Leonardo Tórtoro Pereira (DEMAC/UNESP). **Sistemas de Banco de Dados- Parte 01.**
- PAVLO, Andy; PATEL, Jinglesh (Carnegie Mellon University)**. Databse Systems (15-445).** https://15445.courses.cs.cmu.edu/spring2026/slides/01-relationalmodel.pdf.